



UNIVERSIDAD INTERNACIONAL DEL ECUADOR
ING. CIBERSEGURIDAD - EN LINEA

ASIGNATURA:

LÓGICA DE PROGRAMACIÓN

TEMA:

EVALUACION EN CONTACTO CON EL DOCENTE

CICLO:

PRIMERO

ESTUDIANTE:

ANGEL GABREL YUMBLA OCHOA

DOCENTE:

MONICA PATRICIA SALAZAR TAPIA

FECHA DE ENTREGA:

28 DE JUNIO DEL 2026

1. Introducción

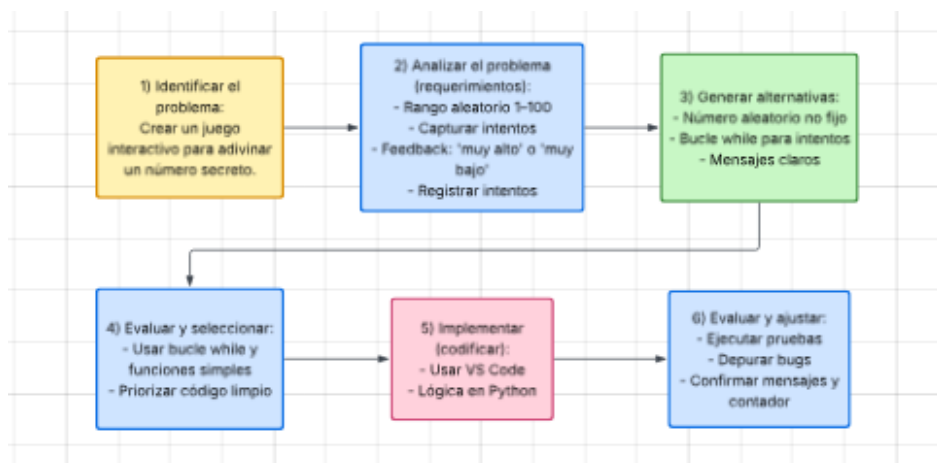
El diseño de algoritmos eficaces y el control estructural de flujos lógicos representan competencias críticas en la Ingeniería en Ciberseguridad. Este documento detalla de manera formal el desarrollo e implementación al 100% del software interactivo "Adivina el Número", programado en el lenguaje Python mediante el entorno de desarrollo Visual Studio Code. A lo largo del informe se expone el proceso de ingeniería de software que permitió transformar un modelo lógico abstracto y secuencial, inicialmente estructurado y validado en diagramas visuales (Raptor), en un sistema robusto, modular y tolerante a fallas operativas, aplicando buenas prácticas de control de estados y consistencia de datos.

2. Descripción del Problema

Este proyecto tiene como objetivo diseñar e implementar un programa interactivo en el lenguaje Python utilizando el entorno de desarrollo VS Code. El problema por resolver se ha estructurado siguiendo una metodología de 6 pasos bien definidos:

1. **Identificación del Problema:** El desafío central consiste en crear un juego interactivo donde el usuario deba adivinar un número secreto generado internamente por la computadora. El propósito es aplicar los conceptos básicos de programación para gestionar la entrada del usuario y la lógica de validación.
2. **Análisis del Problema (Requerimientos):** Para que el juego sea funcional, se han establecido los siguientes requerimientos técnicos:
 - **Rango:** El número secreto debe seleccionarse aleatoriamente en un rango de 1 a 100.
 - **Entrada de Usuario:** El programa debe capturar los intentos del jugador.
 - **Retroalimentación:** Se debe informar al usuario si su intento fue "muy alto" o "muy bajo" con respecto al número secreto.
 - **Intentos:** El juego debe contar y registrar el número total de intentos que el usuario necesita para acertar.
3. **Generación de Alternativas:** Durante esta fase, se evaluaron diferentes enfoques lógicos:
 - Se decidió utilizar un número aleatorio real en lugar de uno fijo para garantizar la Re-jugabilidad.
 - Se seleccionó el uso de un bucle "while" (mientras) como la estructura más eficiente para repetir los intentos de manera indefinida hasta que se encuentre la solución.
 - Se diseñaron mensajes de guía claros para orientar al jugador tras cada intento fallido.

4. **Evaluación y Selección:** Se consolidaron las mejores alternativas técnicas:
 - **Loop "while":** Se validó como la mejor estructura para manejar los intentos repetidos.
 - **Funciones Simples:** Se planea estructurar el código utilizando funciones sencillas para cada tarea (como generar el número, pedir el input y verificar) para asegurar un código limpio y mantenible.
5. **Implementación (Codificación):** Esta etapa implica la traducción de la lógica seleccionada al lenguaje de programación Python. El desarrollo se llevará a cabo dentro del entorno VS Code, escribiendo el código necesario para gestionar las variables, el bucle de intentos, la lógica condicional de comparación y la salida de mensajes en pantalla.
6. **Evaluación y Ajustar:** El paso final consiste en realizar pruebas exhaustivas del juego. Esto incluye:
 - Ejecutar el programa para verificar que el número secreto se genera y adivina correctamente.
 - Probar casos de error (como entradas que no son números) y corregir los "bugs" o errores lógicos.
 - Hay que asegurar que los mensajes de retroalimentación sean precisos y que el contador de intentos funcione de manera fiable.



3. Relación con los Contenidos de la Asignatura

La arquitectura y diseño de este software guarda una relación directa y transversal con las unidades fundamentales de la asignatura de Lógica de Programación:

- **Estructuras de Asignación y Control de Variables:**
Inicialización y almacenamiento dinámico de datos de control de fronteras en variables de estado (rango_minimo, rango_maximo, contador_intentos y respuesta).

Estructuras Repetitivas (Ciclos): Uso de bucles condicionales (while) controlados de manera compuesta por operadores lógicos relacionales, replicando el comportamiento de las estructuras de lazo diseñadas en las herramientas de diagramación.

Estructuras de Selección (Condicionales): Empleo de sentencias lógicas anidadas (if-elif-else) para guiar la bifurcación multitrayecto del programa en respuesta a las cadenas de texto ingresadas.

Programación Modular (Subprocesos): Encapsulamiento de responsabilidades lógicas mediante el uso de funciones (def). Esto plasma de manera tangible una arquitectura de software segmentada por capas (Capa de Presentación e Interfaz, Capa de Validación y Capa de Lógica de Negocio).

4. Explicación del Sistema Desarrollado

El motor algorítmico implementado se fundamenta en la **Búsqueda Binaria**, un método de optimización matemática de orden logarítmico $O(\log n)$. Al calcular el punto medio matemático exacto mediante una división entera $((\text{rango_minimo} + \text{rango_maximo}) // 2)$, el software descarta de forma sistemática el 50% de las opciones incorrectas en cada ciclo con base en la retroalimentación del operador.

El programa está integrado por tres componentes modulares interconectados en Python:

1. **mostrar_bienvenida()** [**Capa de Presentación**]: Controla los flujos de salida en la terminal, formateando las instrucciones operativas claras para guiar al usuario.
2. **obtener_respuesta_valida()** [**Capa de Validación**]: Funciona como un escudo lógico de protección. Normaliza los datos capturados eliminando espacios vacíos y forzando minúsculas (`.strip().lower()`), asegurando la integridad del dato antes de transferirlo a las condicionales primarias.
3. **ejecutar_logica_juego()** [**Capa de Lógica y Datos**]: Orquesta el bucle, gestiona los incrementos aritméticos en el contador de intentos y evalúa en tiempo real si el límite mínimo rebasa al máximo ($\text{rango_minimo} > \text{rango_maximo}$). De cumplirse esta condición, detiene el flujo de forma segura y despliega el aviso formal por error de contradicción del usuario.

5. Reflexión sobre el Impacto de la Tecnología

El desarrollo guiado de este proyecto permite concluir cómo algoritmos optimizados y bien estructurados impactan de forma contundente en el diseño de tecnologías estables y seguras. En el

ámbito de la ciberseguridad, competencias como la segmentación funcional del código, la validación estricta de las entradas y el control milimétrico de las condiciones de parada de los bucles son esenciales para prevenir vulnerabilidades severas, tales como la denegación de servicio lógica (DoS) por agotamiento de recursos del procesador. Diseñar software aplicando principios lógicos rigurosos garantiza que los sistemas automatizados modernos operen bajo entornos predecibles, seguros y eficientes ante la manipulación de datos en entornos digitales.

CRONOGRAMA DE ACTIVIDADES

Semana	Temas de Estudio (Comprensión)	Actividad Práctica (Entrega)	Hitos del Proyecto (Desarrollo)
1	Unidad 1. TEMA 1 - Introducción a la Resolución de Problemas	Práctico 1 (17 mayo)	Inicio: Elección del juego.
2	Unidad 1. TEMA 2 - Entorno de Programación	Práctico 2 (24 mayo)	Avance: Definición de Casos de Uso.
3	Unidad 2. TEMA 1 - Manejo de Datos	Práctico 3 (31 mayo)	Autónomo 1 (31 mayo): Presentación del diagrama de caso de uso.
4	Unidad 2. TEMA 2 - Algoritmos y Diagramas de Flujo	Práctico 4 (07 junio)	Avance: Diagrama de Flujo.
5	Unidad 3. TEMA 1 - Condicionales	Práctico 5 (14 junio)	Integración lógica condicional.
6	Unidad 3. Tema 2 - Bucles	Práctico 6 (21 junio)	Autónomo 2 (21 junio) (Código al 50%): Presentación del diagrama de flujo y avance del código al 50%
7	Unidad 4. TEMA 1 - Estructura de Datos y Funciones	Práctico 7 (28 junio)	Evaluación con Docente (28 junio) (Código final).

Conclusión

A lo largo de este periodo académico, la culminación del proyecto "Adivina el Número" representa la materialización de un proceso estructurado de aprendizaje en el ámbito de la ingeniería en ciberseguridad. Desde la primera semana, el enfoque se centró en comprender la naturaleza de los problemas y la importancia de seguir una metodología rigurosa para abordarlos mediante la computación, lo cual sentó las bases necesarias para organizar las ideas de manera lógica. La transición hacia el manejo de datos, algoritmos y diagramas de flujo permitió visualizar el comportamiento del programa antes de escribir una sola línea de código, garantizando una planificación sólida. A medida que avanzamos, la integración de conceptos clave como estructuras condicionales, bucles, y posteriormente, el uso de funciones y estructuras de datos más complejas transformó una idea inicial en una herramienta funcional y eficiente. El cumplimiento de los hitos establecidos en los trabajos autónomos y los aprendizajes prácticos fue crucial para mantener un ritmo de desarrollo constante, permitiendo alcanzar el avance del 50% y finalmente la culminación del código para la evaluación con el docente. Este recorrido no solo fortaleció habilidades técnicas en Python y en el manejo del entorno VS Code, sino que también consolidó la capacidad analítica para desglosar requerimientos y depurar errores de manera efectiva. En conclusión, la experiencia de desarrollar este juego ha sido un ejercicio integral que vincula la teoría con la práctica aplicada, demostrando que la disciplina en cada etapa es fundamental para lograr un resultado final satisfactorio y funcional.